



yarg-song-server

A self-hosted song library server for [YARG](#) (Yet Another Rhythm Game).

Point it at a folder of songs, run it on whatever is always-on in your house, and every YARG machine on the network plays from the same library. It is a single static binary — Docker, Raspberry Pi, macOS and Windows all run the same code.

Status: the format library works; the server does not exist yet.

You can scan a song library, read and write `.sng` archives, and get an accurate instrument and difficulty grid for every song. There is no HTTP API yet — that is Phase 2. See [docs/ROADMAP.md](#).

What "works" means here: archives written by this tool are decoded by the reference `SngCli` and scanned by a real YARG install, and on a 22-case corpus of deliberately awkward songs, every song YARG rejects is one this scanner independently flags. Details in [docs/TEST-CORPUS.md](#).

Design in one paragraph

The server is a **content source, not a game modification**. It serves ordinary `.sng` files that an unmodified YARG already knows how to read, so the first useful version needs no client changes at all — a small sync companion pulls the library into a normal songs folder. Native in-client browsing comes later, and separately, as an upstream conversation.

Songs are identified by `SHA1(chart file bytes)`, which is exactly how YARG itself identifies them. Client and server therefore agree precisely on "do I already have this song", with no heuristics and no fuzzy matching.

What it will and will not do

Will

- Ingest loose song folders (`song.ini` + `notes.mid` / `notes.chart` + stems), `.sng` containers, and zipped versions of either
- Serve a browsable, searchable catalog over HTTP, sorted by the same attributes YARG sorts by
- Serve songs as `.sng`, and answer "which of these hashes am I missing?" in bulk
- Run on `linux/amd64`, `linux/arm64` (Raspberry Pi), macOS and Windows

Will not, ever

- Decrypt Rock Band CON packages or encrypted moggs. Upstream lists CON decryption as out of scope and will reject PRs for it on sight; it also carries real legal exposure. `.con`, `_rb3con` and `.pkg` are refused on ingest with a clear message.
- Generate YARG's `songcache.bin`. Not because it is unreadable — it is a plain binary file — but because it stores **absolute local paths**, so a cache built here is meaningless on your machine; because its version is a date stamp checked with no compatibility window and no migration path, and its field layout is internal with no version of its own; and because getting any of it wrong fails *silently*, with YARG quietly rebuilding the cache while the tool appears to have worked.
- Distribute copyrighted audio. Charts and audio are separable by design.

Documentation

Document	What it covers
docs/ROADMAP.md	Phases, build order, exit criteria, blockers
docs/ADR-001-server-architecture.md	Why Go, why two repos, why sync-first, why LGPL
docs/research/yarg-song-formats.md	The <code>.sng</code> binary layout, <code>song.ini</code> keys, the metadata model, song identity, and a difficulty assessment for every part of a Go reimplementation

Read the research document before writing any parser code. It is the reason the scope is what it is.

Building

```
make build      # host binary into ./bin
make test       # unit tests
make docker     # multi-arch image
```

Trying the scanner

There is no server yet, but the scanner works. Point it at a song library and it prints the catalog as JSON, one song per record:

```
./bin/yarg-song-server scan /path/to/songs
```

It recognises loose song folders and `.sng` archives, walks nested directories, skips folders that hold no chart, and reports an unreadable song on stderr rather than abandoning the scan.

Repacking a loose folder into a `.sng` also works:

```
./bin/yarg-song-server pack /path/to/song-folder out.sng
```

The chart is copied byte for byte, so the song's identity is unchanged. Archives written this way have been decoded by the reference `SngCli` and scanned by a real YARG install — see [docs/TEST-CORPUS.md](#).

Licence

LGPL-3.0-or-later, matching YARG and YARG.Core. See [LICENSE](#).

This project is not affiliated with or endorsed by YARC. It stands with upstream against piracy: it ships no content, and it will not help you play content you do not own.